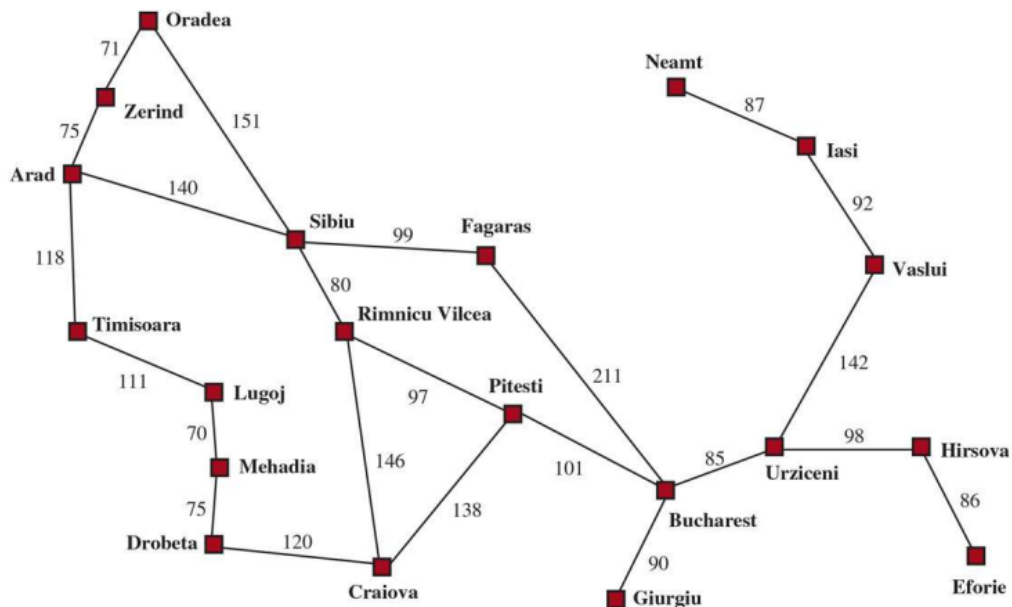

CHAPTER 3

INFORMED SEARCH ALGORITHMS

Note: This lecture closely follows Chapter 3.6 (Heuristic Functions), 3.5 (Informed Search) to 3.5.1 (Greedy Best First Search), 3.5.2 to 3.5.4 (A* Search), 3.6.1 (Effect of Heuristic on accuracy and performance) of Russel and Norvig, *Artificial Intelligence: A Modern Approach*. The images are also borrowed from these chapters

As computer science students, you are already familiar with various search algorithms such as Breadth-First Search, Depth-First Search, and Dijkstra's/Best-First Search. These strategies fall under the category of Uninformed Search or Blind Search, which means they rely solely on the information provided in the problem definition.



A simplified road map of part of Romania, with road distances in miles.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

For example, consider a map of Romania where we want to travel from Arad to Bucharest. The map indicates that Arad is connected to Zerind by 75 miles, Sibiu by 140 miles, and Timișoara by 118 miles. Using a blind search strategy, the next action from Arad would be chosen based solely on the distances to these connected cities. This approach can be slower and less efficient as it may explore paths that are irrelevant to reaching the goal efficiently.

In this course, we will focus on informed search strategies, also known as heuristic search. Informed Search uses additional information—referred to as heuristics—to make educated guesses about the most promising direction to pursue in the search space. This approach often results in faster and more efficient solutions because it avoids wasting time on less likely paths. We will study Greedy Best-First Search and A* search extensively. But first, let's explore the concept of heuristics.

3.1 Heuristic Function

In the context of informed search algorithms, a heuristic is a technique that helps the algorithm estimate the cost (often the shortest path or least costly path) from a current state (or node) to the goal state. It's essentially a function that provides guidance on which direction the search should take in order to find the most efficient path to the goal. This guidance allows informed search algorithms to perform more efficiently than uninformed search algorithms, which do not have knowledge of the goal state as they make their decisions.

3.1.1 Key Characteristics of Heuristics in Search Algorithms

Estimation: A heuristic function estimates the cost to reach the goal from a current node. This estimate does not need to be exact but should never overestimate.

Returning to the example of traveling to Bucharest from Arad: A heuristic function can estimate the shortest distance from any city in Romania to the goal. For instance, we might use the straight-line distance as a measure of the heuristic value for a city. The straight-line distance from Arad to Bucharest is 366 miles, although the optimal path from Arad to

Bucharest actually spans 418 miles. Therefore, the heuristic value for Arad is 366 miles. For each node (in this problem, city) in the state space (in this problem, the map of Romania) the heuristic value will be their straight line distance from the goal state (in this case Bucharest).

3.1.2 Why do we use heuristics?

Guidance: The heuristic guides the search process, helping the algorithm prioritize which nodes to explore next based on which seem most promising—i.e., likely to lead to the goal with the least cost.

Efficiency: By providing a way to estimate the distance to the goal, heuristics can significantly speed up the search process, as they allow the algorithm to focus on more promising paths and potentially disregard paths that are unlikely to be efficient.

3.2 Heuristic functions to solve different problems

Generating a heuristic function for use in informed search algorithms involves a process where the function must effectively estimate the cost, usually the shortest path, from any node or state in the search space to the goal. The design of the heuristic function is based on the problem domain, which requires an understanding of the rules, constraints, and ultimate goal of the problem. Here are some examples of heuristic functions for different types of problems.

8-Puzzle Game

The number of misplaced tiles (blank not included): For the figure above, all eight tiles are out of position, so the start state has $h_1 = 8$.

The sum of the distances of the tiles from their goal positions: Because tiles cannot move along diagonals, the distance is the sum of the horizontal and vertical distances—sometimes called the city-block distance or Manhattan distance.

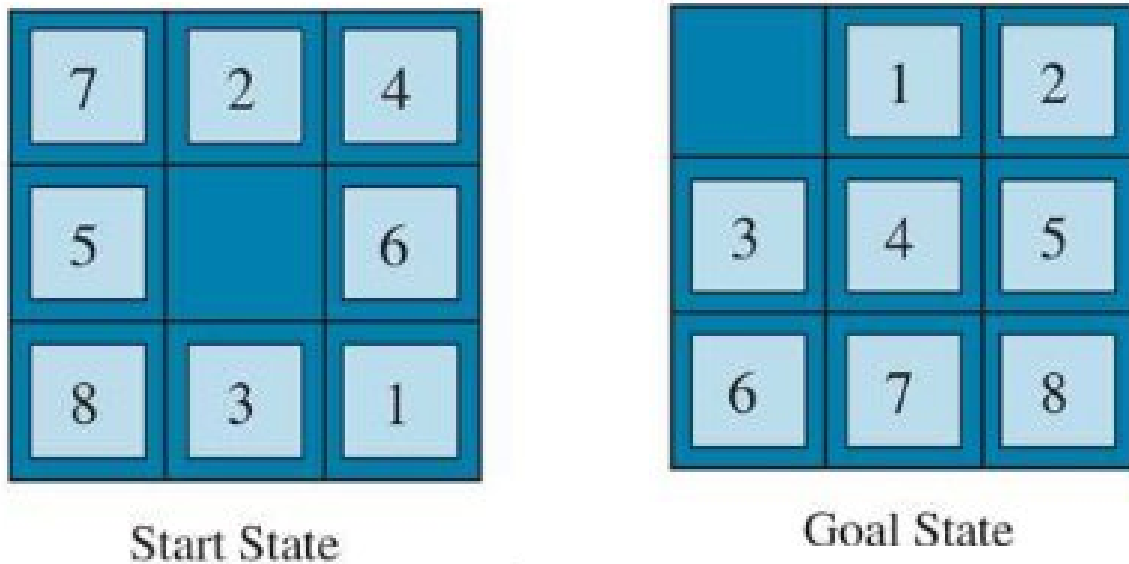


Figure 3.1: Initial and Goal State of a 8-Puzzle Game (Russel and Norvig, *Artificial Intelligence: A Modern Approach*)

Pathfinding in Maps and GPS Systems

Straight-line Distance: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ where (x_1, y_1) and (x_2, y_2) are the coordinates of the current position and the goal.

Travel Time: Estimating the time needed to reach the goal based on average speeds and road types. $t = \frac{d}{v}$ where d is the distance and v is the average speed.

Traffic Patterns: Using historical or real-time traffic data to estimate the fastest route. Could involve a weighting factor, w based on traffic data, modifying the travel time: $t_{adjusted} = t \times w$.

Game AI (e.g., Chess, Go)

Material Count: Sum of the values of all pieces. For example, in chess, pawns = 1, knights/bishops = 3, rooks = 5, queen = 9.

Positional Advantage: A score based on piece positions. E.g., control of center squares in chess might be given additional points.

Mobility: Number of legal moves available M for a player at a given turn.

Web Search Engines

Keyword Frequency: The number of times a search term appears on a web-page.

$$F = \frac{\text{Number of occurrences of keyword}}{\text{Total number of words in document}}$$

Page Rank: Evaluating the number and quality of inbound links to estimate the page's importance.

Domain Authority: The reputation and reliability of the website hosting the information. Often a proprietary metric, but generally a combination of factors like link profile, site age, traffic, etc.

Robotics and Path Planning

Distance to Goal: Estimating the remaining distance to the target location. Same as straight-line distance in GPS systems.

Obstacle Proximity: Distance to the nearest obstacle to avoid collisions. $O = \min(\text{distance to each obstacle})$.

Energy Efficiency: Estimating the most energy-efficient path, important for battery-powered robots.

$$E = \sum \text{energy per unit distance} \times \text{path distance}$$

Natural Language Processing (NLP)

Word Probability: $P(w \mid \text{context})$ where w is the word and context represents the surrounding words.

Semantic Similarity: How closely words or phrases match in meaning. Often uses cosine similarity,

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

where (A) , and (B) are vector representations of words or sentences.

Language Consistency: Ensuring the text follows grammatical and syntactical norms of the target language. Can be quantified using perplexity in language models.

Recommendation Systems User Behavior Tracking: Score items based on frequency and recency of user interactions. Analyzing past purchases or viewing habits to predict future interests.

Item Similarity: Recommending products similar to those a user has liked or purchased. Cosine similarity or other distance measures between item feature vectors.

Collaborative Filtering: Using preferences of similar users to recommend items. Matrix factorization techniques or neighbor-based algorithms to predict user preferences.

3.3 Greedy Best First Search- Finally an algorithm

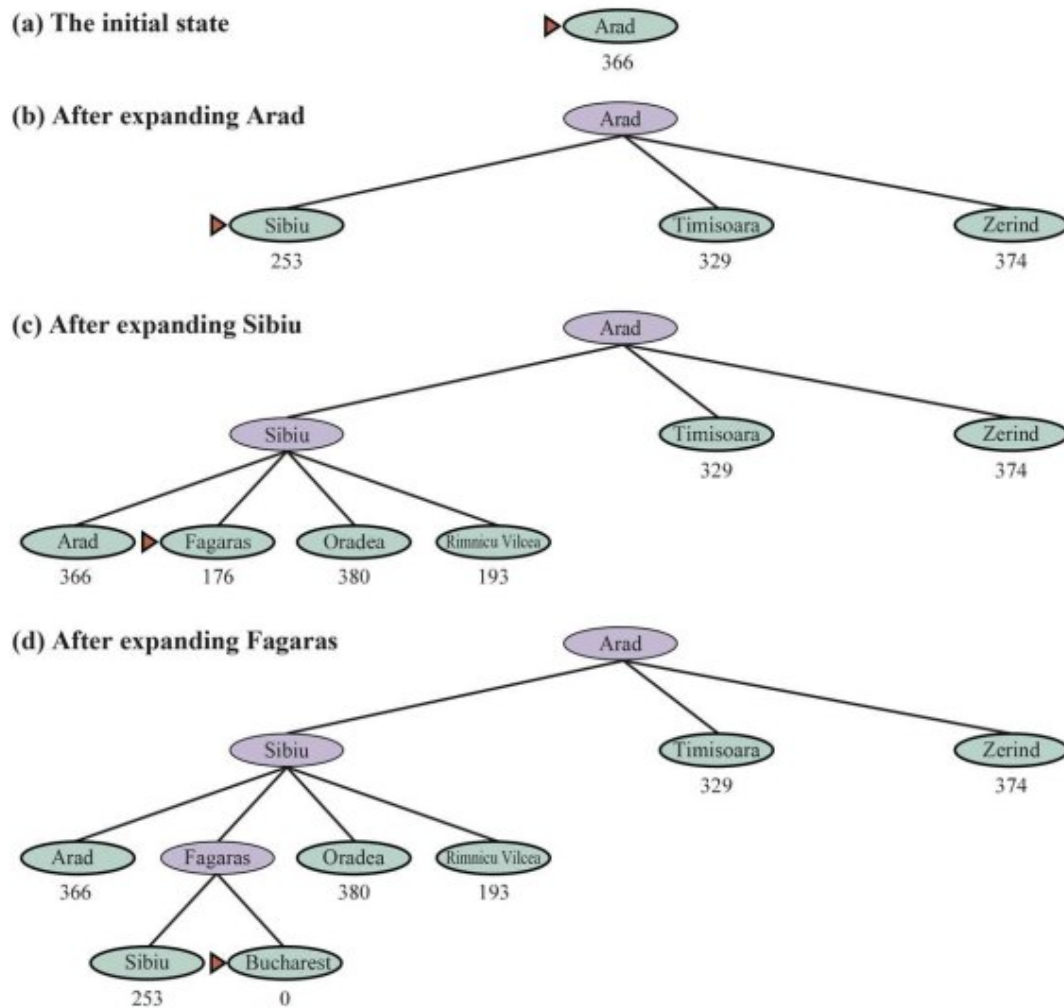
Greedy best-first search is a form of best-first search that expands first the node with the lowest $h(n)$ value—the node that appears to be closest to the goal—on the grounds that this is likely to lead to a solution quickly. So the evaluation function $f(n) = h(n)$.

Let us see how this works for route-finding problems in Romania; we use the **straight-line-distance** heuristic, which we will call h_{SLD} . If the goal is Bucharest, we need to know the straight-line distances to Bucharest, which are shown in the figure below. For example, $h_{SLD}(Arad) = 366$. Notice that the values of h_{SLD} cannot be computed from the problem description itself (that is, the **ACTIONS** and **RESULT** functions). Moreover, it takes a certain amount of world knowledge to know that h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic.

The next figure shows the progress of a greedy best-first search using h_{SLD} to find a path from Arad to Bucharest. The first node to be expanded from Arad will be Sibiu because the heuristic says it is closer to Bucharest than is either Zerind or Timisoara. The next node to be expanded will be Fagaras because it is now closest according to the heuristic. Fagaras in turn generates Bucharest, which is the goal. For this particular problem, greedy best-first search using h_{SLD} finds a solution without ever expanding a node that is not on the solution path. The solution it found does not have optimal cost, however: the path via Sibiu and Fagaras to Bucharest is 32 miles longer than the path through Rimnicu Vilcea and Pitesti. This is why the algorithm is called “greedy”—on each iteration it tries to get as close to a goal as it can, but greediness can lead to worse results than being careful.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.



Stages in a greedy best-first tree-like search for Bucharest with the straight-line distance heuristic h_{SLD} . Nodes are labeled with their h -values.

Greedy best-first graph search is complete in finite state spaces, but not in infinite ones. The worst-case time and space complexity is $O(|V|)$. With a good heuristic function, however, the complexity can be reduced substantially, on certain problems reaching $O(bm)$.

3.3.1 Algorithm: Greedy Best-First Search

Algorithm 1 Greedy Best First Search Algorithm

- Input:
 - **start**: The target node of the search
 - **goal**: The target node to reach
 - **heuristic(node)**: A function that estimates the cost from node to the goal
 - Output:
 - The path from **start** to **goal** if one exists, otherwise **None**.
 - Procedure
 - Initialize:
 - Create a priority queue and insert the start node along with its heuristic value **heuristic(start)**.
 - Define a **visited** set to keep track of all visited nodes to avoid cycles and redundant paths.
 - Search:
 - While the priority queue is not empty:
 - * Remove the node **current** with the lowest heuristic value from the priority queue.
 - * If **current** is the goal, return the path that led to current.
 - * Add **current** to the **visited** set.
 - * For each neighbor **n** of **current**:
 - If **n** is not in **visited**:
 - Calculate the heuristic value **heuristic(n)**.
 - Add **n** to the priority queue with the priority set to **heuristic(n)**.
 - Failure to find the goal:
 - If the priority queue is exhausted without finding the **goal**, return **None**.
-

3.3.2 Key Points

Heuristic Function: This function is crucial as it determines the search behavior. A good heuristic can dramatically increase the efficiency of the search.

Completeness and Optimality: Greedy Best-First Search does not guarantee that the shortest path will be found, making it neither complete nor optimal. It can get stuck in loops or dead ends if not careful with the management of the visited set.

Data Structures: The algorithm typically uses a priority queue for the frontier and a set for the visited nodes. This setup helps in efficiently managing the nodes during the search process.

Greedy Best-First Search is particularly useful when the path's exact length is less important than quickly finding a path that is reasonably close to the shortest possible. **It is well-suited for problems where a good heuristic is available.**

3.4 A* Search algorithm

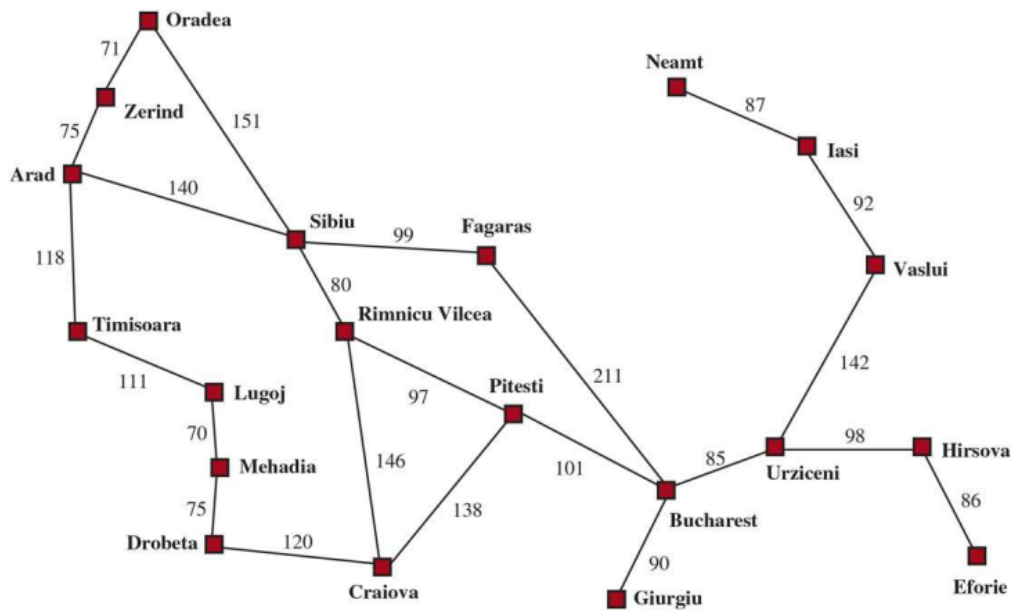
The most common informed search algorithm is A* Search (pronounced "A-star search"), a best-first search that uses the evaluation function $f(n) = g(n) + h(n)$ where $g(n)$ is the path cost from the initial state to node n , and $h(n)$ is the estimated cost of the shortest path from n to a goal state, so we have $f(n)$ estimated cost of the best path that continues from n to a goal.

3.4.1 Algorithm: A* Search

Algorithm 2 A* Search Algorithm

- **Input:**
 - **start**: The starting node of the search.
 - **goal**: The target node to reach.
 - **neighbors(node)**: A function that returns the neighbors of **node**.
 - **cost(current, neighbor)**: A function that returns the cost of moving from **current** to **neighbor**.
 - **heuristic(node)**: A function that estimates the cost from **node** to the **goal**.
- **Output:**
 - The path from **start** to **goal** if one exists, otherwise **None**.
- **Procedure**
 - Initialize
 - Create a priority queue and insert the **start** node.

- Set `gScore[start]` to 0 (cost from `start` to `start`).
 - Set `fScore[start] = 0 + heuristic(start)` (total estimated cost from `start` to goal through `start`).
 - Define `cameFrom` to store the path reconstruction data.
 - Search:
 - While the priority queue is not empty:
 - * Remove the node `current` with the lowest `fScore` value from the priority queue.
 - * If `current` is the goal, reconstruct and return the path from `start` to goal using `cameFrom`.
 - * For each neighbor of `current`:
 - * For each neighbor of `current`:
 - Calculate `tentative-gScore` as `gScore[current] + cost(current, neighbor)`.
 - If `tentative-gScore` is less than `gScore[neighbor]` (or `neighbor` is not in `priority queue`):
 - Update `cameFrom[neighbor]` to `current`.
 - Update `gScore[neighbor]` to `tentative-gScore`.
 - Update `fScore[neighbor]` to `tentative-gScore + heuristic(neighbor)`.
 - If `neighbor` is not in the priority queue, add it.
 - Failure to find the goal:
 - If the priority queue is exhausted without reaching the `goal`, return `None`.
-



A simplified road map of part of Romania, with road distances in miles.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

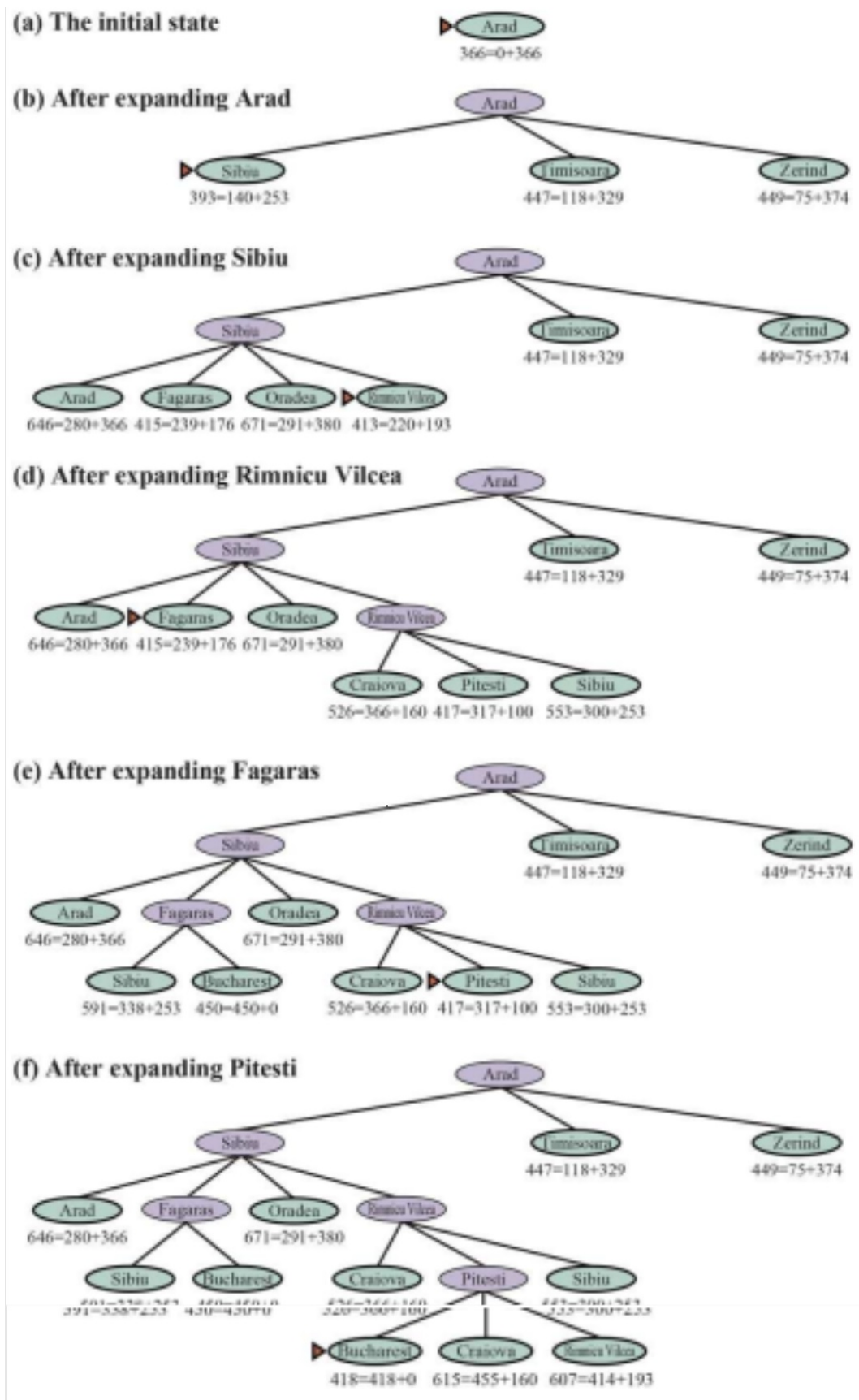


Figure 3.2: Simulation of A* Algorithm to find a path from Arad to Bucharest.

Path Reconstruction: The path is reconstructed from the `cameFrom` map, which records where each node was reached from.

Notice that Bucharest first appears on the frontier at step (e), but isn't selected for expansion as it isn't the lowest-cost node at that moment, with a cost of 450 compared to Pitesti's lower cost of 417. The algorithm prioritizes exploring potentially cheaper routes, such as through Pitesti, before settling on higher-cost paths. By step (f), a more cost-effective path to Bucharest, costing 417, becomes available and is subsequently selected as the optimal solution.

So we can say that the A* algorithm has the following properties,

- **Cost Focus:** Prioritizes nodes with the lowest estimated total cost, $f(n) = g(n) + h(n)$
- **Guarantees Optimality:** Ensures the solution is the least expensive by expanding the cheapest node first.
- **Resource Efficiency:** Avoids exploring more expensive paths when cheaper options are available.
- **Adapts Based on New Information:** Adjusts path choices dynamically as new cost information becomes available.
- **Heuristic Importance:** Relies on the heuristic to guide the search efficiently by estimating costs.

A* Algorithm is always complete, meaning that if a solution exists then the algorithm will find the path.

However, the A* algorithm only returns optimal solutions when the heuristic has some specific properties.

3.5 Condition on heuristics for A* to be optimal:

Whether A* Search is optimal depends on two key properties of the heuristic. These are **Admissibility** and **Consistency**.

Definition 3.5.1: Admissibility

An admissible heuristic never overestimates the cost to reach the goal. This makes it optimistic about the path costs.

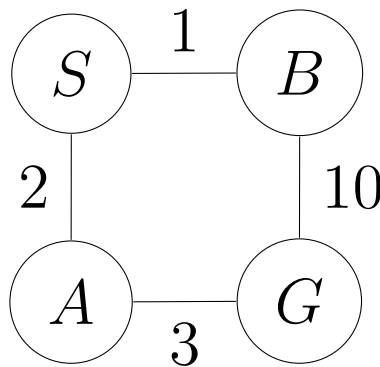
Proof of Cost-Optimality (via Contradiction)

- Assume the optimal path cost is C^* but A* returns a path with a cost greater than C^* .
- There must be a node n on the optimal path that A* did not expand.

- If $f(n)$ which is the estimated cost of the cheapest solution through n were less or equal to C^* then n would have been expanded.
- By definition and admissibility, $f(n)$ is $g(n) + h(n)$ and should be less or equal to $g^*(n) + \text{cost}(n \text{ to goal}) = C^*$ as n is on the optimal path and $h(n)$ is less than or equal $\text{cost}(n \text{ to goal})$ due to admissibility
- This is contradicting our assumption that $f(n) > C^*$.
- Thus, A^* must indeed return the cost-optimal path.

Example: Where Violation of admissibility Leads to a Suboptimal Solution

- **Nodes:** Start (S), A, B, Goal (G)
- **Edges with costs:**
 - S to $A = 2$
 - A to $G = 3$
 - S to $B = 1$
 - B to $G = 10$



Heuristic, $h(\text{node})$ Estimates to the Goal, G :

- $h(S) = 3$ admissible as $h(S) < \text{the optimal path-cost from } S \text{ to } G$.
- $h(A) = 10$ is in-admissible as $h(A) > \text{optimal path-cost from } A \text{ to } G$
- $h(B) = 2$ admissible.
- $h(G) = 0$ admissible.

A* Algorithm Execution:

- **Start at S :**

$$f(S) = g(S) + h(S) = 0 + 3 = 3.$$

- **Expand S:** Adding Neighbors (A and B) to the queue:

– For A:

$$g(A) = 2, \quad \text{so, } f(A) = g(A) + h(A) = 2 + 10 = 12.$$

– For B:

$$g(B) = 1, \quad \text{so, } f(B) = g(B) + h(B) = 1 + 2 = 3$$

- Node A and Node B are currently in queue.
- **Node B selected** from the queue for expansion because of **lower f-value** despite leading to a higher cost path.
- **Expand B:**, Adding Neighbor (G):
 - For G via B

$$g(G) = 11, \quad \text{so, } f(G) = g(G) + h(G) = 11 + 0 = 11$$

- Node A and Node G are currently in queue.
- **Node G selected** from the queue for expansion because of lower f-value.
- The algorithm returns path S-B-G which is sub-optimal.

The inadmissibility of the the heuristic causes the algorithm to prefer a sub-optimal path.

Definition 3.5.2: Consistency

A heuristic h is consistent if for every node n and every successor n' of n , the estimated cost from n to the goal, denoted $h(n)$, does not exceed the cost from n to n' plus the estimated cost from n' to the goal, $h(n')$. Mathematically, this is represented as: $h(n) \leq c(n, n') + h(n')$ where $c(n, n')$ is the cost to reach n' from n .

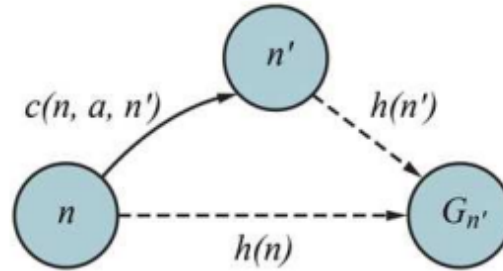
A consistent heuristic, also known as a monotonic heuristic, is a stronger condition than admissibility and plays a crucial role in ensuring that A* search finds the optimal path. Here's how it ensures that A* returns an optimal path:

1. Path Cost Non-Decreasing:

- Consistency ensures that the f-value (total estimated cost) of a node n calculated as $f(n) = g(n) + h(n)$ does not decrease as the algorithm progresses from the start node to the goal. This is because for any node n and its successor n' : $g(n') = g(n) + c(n, n')$
- Simplifying, we find:

$$f(n) = g(n) + h(n) \leq g(n) + c(n, n') + h(n') = g(n') + h(n') = f(n')$$

- Therefore, f-values along a path do not decrease, preventing any re-exploration of nodes already deemed sub-optimal, hence streamlining the search towards the goal.



Triangle inequality: If the heuristic h is **consistent**, then the single number $h(n)$ will be less than the sum of the cost $c(n, a, a')$ of the action from n to n' plus the heuristic estimate $h(n')$.

2. Closed Set Invariance:

- When a node n is expanded, its f-value is finalized. Due to the non-decreasing nature of f-values, any path rediscovered through n will have an f-value at least as large as when n was first expanded.
- This prevents the algorithm from revisiting nodes unnecessarily, thereby ensuring efficiency in path finding.

3. Optimal Path Discovery:

- Given the consistency condition, once the goal node g is reached and its $f(g)$ calculated, there can be no other path to g with a lower f-value that has not already been considered.
- Since $h(g) = 0$ (by definition at the goal), $f(g) = g(g)$, meaning that the path cost $g(g)$ represents the total minimal cost to reach the goal from the start node.

4. Optimality Guarantee:

- The search terminates when the goal is popped from the priority queue for expansion, and due to the non-decreasing nature of f-values, this means that no other path with a lower cost can exist that has not already been evaluated.
- Thus, the path to g with cost $g(g)$ must be optimal.

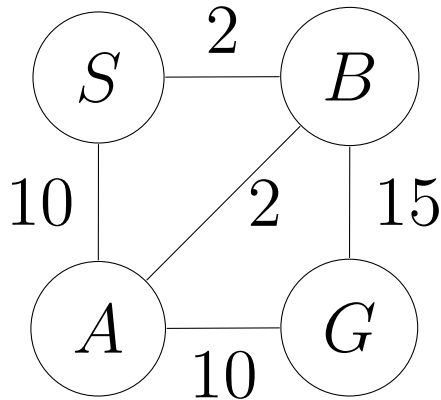
By adhering to these principles, A* search with a consistent heuristic not only finds a solution but ensures it is the optimal one.

Example: Checking inconsistency

- **Nodes:** Start (S), A, B, Goal (G)

- **Edges with costs:**

- S to $A = 10$
- A to $G = 10$
- S to $B = 2$
- B to $G = 15$
- A to $B = 2$



- **Heuristic (h) Estimates to the Goal (G):**

- $h(S) = 12$
- $h(A) = 7$
- $h(B) = 10$
- $h(G) = 0$

- **Checking consistency for each node:** First we find the optimal path to Goal from each node.

- Optimal path from Node S is $S - B - A - G = 14$.
- Optimal path from Node B is $B - A - G = 12$.
- Optimal path from Node A is $A - G = 10$.
- Optimal path from Node G is $G - = 0$.

For any given node, n the heuristic, $h(n)$ for that node is lower or equal than the optimal path-cost from n . So we can say that the given heuristics are admissible. An inadmissible heuristic automatically leads to inconsistent heuristic. But admissible heuristic does not guarantee consistency.

It is better to start checking consistency from the Goal node.

- We see $h(G) = 0$ is admissible and consistent.
- Next, from Node A , there is a direct path to Goal node, G with cost 10 which is the optimal path. There is also a path via B . So to be consistent,

$$\begin{aligned} h(A) &\leq \text{Cost}(A, G) + h(G) \\ &= 10 + 0 = 10 \\ h(A) &= 7 \end{aligned}$$

So, node A is consistent with node G .

- Next, from Node B , there is a direct path to Goal node, G with cost 15 and a path via node A costing 12. In this case, A is the successor of B . So to be consistent,

$$\begin{aligned} h(B) &\leq \text{Cost}(B, A) + h(A) \\ &= 2 + 7 = 9 \\ h(B) &= 10 \end{aligned}$$

So, heuristic of node B with node A is inconsistent. Similarly for node S ,

$$\begin{aligned} h(S) &\leq \text{cost}(S, B) + h(B) = 12 \\ h(S) &= 12 \end{aligned}$$

Making, $h(S)$ consistent with node B .

3.6 How to choose a better Heuristic:

In the previous lecture, we have seen that there are many possible ways to design the heuristics for a problem space. We want to know which heuristic function should be selected when we have more than one way of computing admissible and consistent heuristics.

1. **Effective Branching Factor:** The effective branching factor (**EBF**) is a measure used in tree search algorithms to provide a quantitative description of the tree's growth rate. It reflects how many children each node has, on average, in the search tree that needs to be generated to find a solution. Mathematically, the EBF is defined as the branching factor b for which:

$$N + 1 = 1 + b + b^2 + b^3 + \dots + b^d$$

where N is the total number of nodes generated in the search tree and d is the depth of the shallowest solution.

The EBF gives an insight into the efficiency of the search process, influenced heavily by the heuristic used:

- **Lower EBF:** A lower EBF suggests that the heuristic is effective, as it leads to fewer nodes being expanded. This usually indicates a more directed and efficient search.

- **Higher EBF:** A higher EBF suggests a less effective heuristic, as more nodes are being generated, indicating a broader search, which is generally less efficient.

Calculating the EBF can help evaluate the practical performance of a heuristic. An ideal heuristic would reduce the EBF to the minimum necessary to find the optimal solution, indicating a highly efficient search strategy.

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Comparison of the search costs and effective branching factors for 8-puzzle problems using breadth-first search, A^* with

h_1

(misplaced tiles), and A^* with

h_2

(Manhattan distance). Data are averaged over 100 puzzles for each solution length

d

from 6 to 28.

In the above figure, Stuart and Russel generated random 8-puzzle problems and solved them with an uninformed breadth-first search and with A^* search using both and reporting the average number of nodes generated and the corresponding effective branching factor for each search strategy and for each solution length. The results suggest that h_2 is better than h_1 and both are better than no heuristic at all.

2. **Dominating heuristic:** For two heuristic functions, h_1 and h_2 , we say that h_1 dominates h_2 if for every node n in the search space, the following condition holds:

$h_1(n) \geq h_2(n)$ and there is at least one node n where $h_1(n) > h_2(n)$ then we say that h_1 dominates h_2 or in other words, h_1 is the dominating heuristic.

3.6.1 Why is a dominant heuristic for efficient?

By now we should have an understanding that in A^* algorithm every node, n with

$$f(n) \leq C^*,$$

or, $h(n) \leq C^* - g(n)$ [C^* is optimal path cost from initial node via n]

is surely expanded.

Now let us consider, two heuristics, h_1 and h_2 , both admissible and consistent. Where,

$$h_2 \geq h_1.$$

If node, n is on the optimal path, it will be expanded with A* algorithm for both the heuristics. Meaning,

$$h_1(n) \leq h_2 \leq C^* - g(n).$$

If node, n is not on the optimal path, we have three possibilities,

- **Possibility 1:**

$$C^* - g(n) \leq h_1 \leq h_2,$$

in which case, node n will not be expanded at all, whichever heuristic we use.

- **Possibility 2:**

$$h_1(n) \leq h_2 \leq C^* - g(n).$$

in this case, node, n will be expanded for both the heuristics.

- **Possibility 3:**

$$h_1(n) \leq C^* - g(n) \leq h_2(n),$$

where, node n will be expanded when h_1 is used but not when h_2 is used.

So, we see that the using h_1 may expand the same nodes that are expanded when h_2 is used. But it may end up expanding more unnecessary nodes than those expanded by using the dominant heuristic h_2 .

You may want to go back to the last example in section 3.3 and notice that the number of nodes generated by using the dominant heuristic is significantly less for the 8-puzzle problem.

So, given that the heuristic is consistent and the computation time is not too long, it is generally a better idea to use higher valued heuristic.